

Instituto Politécnico Nacional

Escuela Superior de Cómputo

Trabajo Extra: Clasificación de CIFAR-100

CNN desde cero, Data Augmentation, Transfer Learning y
Fine-Tuning

Redes Neuronales y Aprendizaje Profundo — Unidad 5

Alumno: Said Carbot Cruz Trejo

25 de junio de 2026

1. Resumen general de las dos tareas

Estuvo tardado y más complejo y complicado de lo que pensaba. Primero, pensé en entrenar los modelos en mi computadora, pero gracias a dios algo me dijo – Mejor usa las GPUs de Collab– Nunca las había usado, pero que bueno que lo hice porque entrenar los modelos con GPU me tomó 25 minutos para el más tardado. Fue un desastre porque para cuando quería usar las salidas del collab, por la inactividad se me desconectaba todo y tenía que hacer malabares para no volver a correr todo pero sí sacar las variables, por eso la segunda matriz de confusión no quedo con los datos exactos (hay una variación del 0.04 por ciento) porque son corridas diferentes, pues se me olvido hacer la matriz de confusión la primera vez, pero como se tardaba mucho al final ya solo corrí el modelo nuevamente y genere la matriz de confusión del modelo que ya sabía que tendría un valor de retrieval mejor al umbral del 75 por ciento.

1.1. Comparación Final de Estrategias

Tabla 1: Comparación final entre el mejor modelo entrenado desde cero y el mejor modelo con Transfer Learning y Fine-Tuning.

Mejor modelo	Estrategia	Acc. train	Acc. val	Acc. test	Umbral requerido	¿Supera el umbral?
CNN 3	CNN desde cero	69.40 %	53.50 %	54.90 %	50.00 %	Si
EfficientNetB0 (Protegido)	Transfer Learning + Fine-Tuning	88.70 %	82.43 %	82.58 %	75.00 %	Si

1.2. Primera tarea

Tabla 2: Resultados experimentales y configuraciones de hiperparámetros.

Modelo	Arquitectura resumida	DA	BatchNorm	Dropout	Optimizador	LR	Batch	Epocas	Acc. train	Acc. val	Acc. test
CNN 1	2 Conv (32, 64), 1 Dense (256)	Si	No	No	Adam	0.001	64	30	50.2 %	40.0 %	40.2 %
CNN 2	2 Conv (64, 128), 1 Dense (512)	Si	Si	Si (0.4)	Adam	0.001	64	30	63.9 %	50.3 %	52.1 %
CNN 3	2 Conv (128, 256), 1 Dense (1024)	Si	Si	Si (0.5)	Adam	0.001	64	30	69.4 %	53.5 %	54.9 %

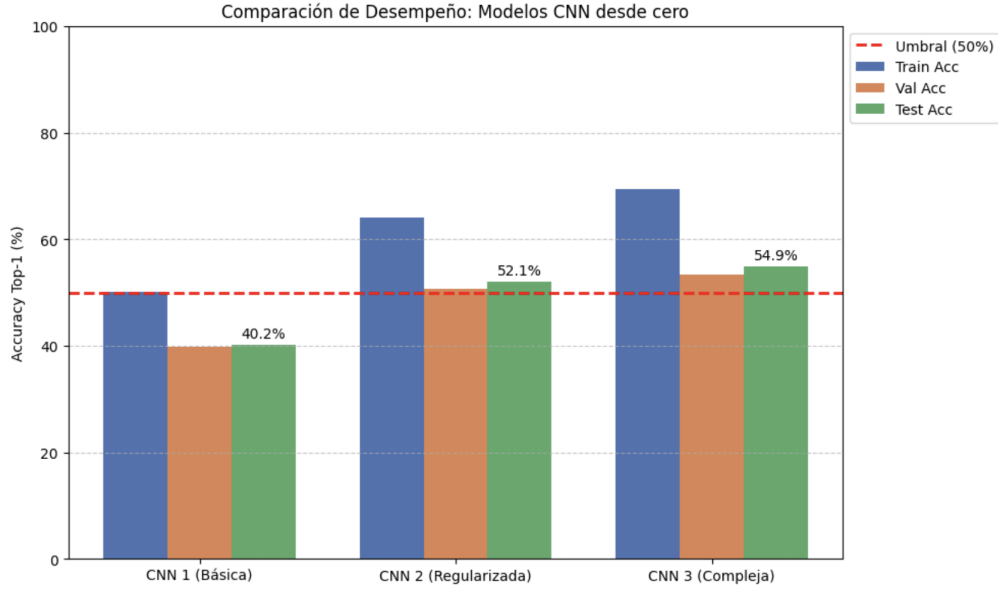


Figura 1: Gráficas de comparación de modelos CNN (Básico, Regularizado y Complejo) para datos de entrenamiento, validación y prueba.

En la gráfica de la Figura 1 de siguiente muestro los tres experimentos que hice para una CNN básica, otra regularizada y una compleja al final. En color azul están los porcentajes de la accuracy para el porcentaje de los datos que se utilizaron para el entrenamiento, en naranja para los datos de validación y en verde para los datos de prueba. Entonces dos modelos, el regularizado y el complejo, superaron el umbral del 50 por ciento para los datos de prueba.

1.3. Segunda tarea

Tabla 3: Resultados globales del proceso iterativo de Transfer Learning (TL) y Fine-Tuning (FT)

Modelo Base	Input	Congeladas	Descongeladas	Opt.	LR TL	LR FT	Batch	Ep. TL	Ep. FT	Acc. train	Acc. val	Acc. test
<i>Corrida Inicial: Configuración Base (8 Épocas de FT)</i>												
TL/FT 1 (MobileNetV2)	128x128	Todo menos 30	30	Adam	1e-3	1e-5	64	6	8	51.72 %	63.93 %	63.12 %
TL/FT 2 (ResNet50V2)	128x128	Todo menos 40	40	Adam	1e-3	1e-5	64	6	8	57.18 %	67.10 %	67.08 %
TL/FT 3 (EfficientNetB0)	128x128	Todo menos 20	20	Adam	1e-3	1e-5	64	6	8	51.35 %	68.04 %	68.10 %
<i>Segunda Corrida: Incremento de Épocas y Capas (16 Épocas de FT)</i>												
TL/FT 1 (MobileNetV2)	128x128	Todo menos 50	50	Adam	1e-3	1e-5	64	6	16	59.41 %	68.25 %	67.61 %
TL/FT 2 (ResNet50V2)	128x128	Todo menos 70	70	Adam	1e-3	1e-5	64	6	16	69.67 %	71.29 %	70.83 %
TL/FT 3 (EfficientNetB0)	128x128	Todo menos 40	40	Adam	1e-3	1e-5	64	6	16	57.26 %	70.75 %	72.50 %
<i>Tercera Corrida: Ajuste Agresivo de Capas Profundas</i>												
TL/FT 3 (EfficientNetB0)	128x128	Todo menos 100	100	Adam	1e-3	5e-5	64	6	30	71.89 %	75.88 %	75.68 %
<i>Cuarta Corrida: Arquitectura de Alta Resolución con BN Protegido</i>												
EfficientNetB0 (Protegido)	160x160	Solo BatchNorm	Todas menos BN	Adam	1e-3	5e-5	64	5	15	88.70 %	82.43 %	82.58 %

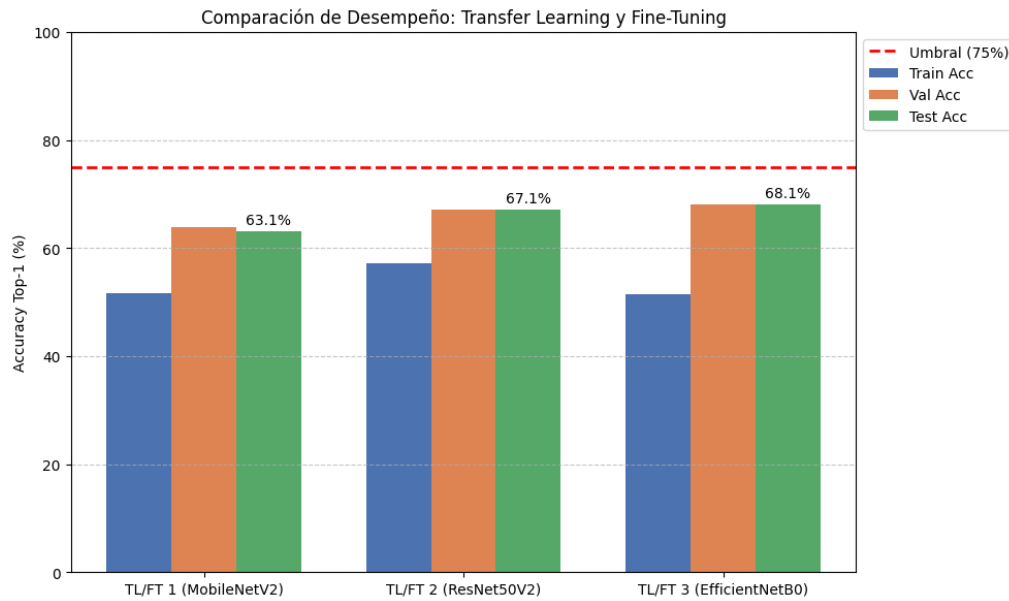


Figura 2: Gráficas de comparación de los tres modelos base para la primera corrida.

Esa parte estuvo tardado y complicada, primero hice una corrida inicial para tres modelos diferentes MobileNetV2, ResNet50V2 y EfficientNetB0. Pero ninguno de los modelos con el Transfer learning y el Fine Tuning pudo superar el umbral del 75 por ciento (Ver figura 2).

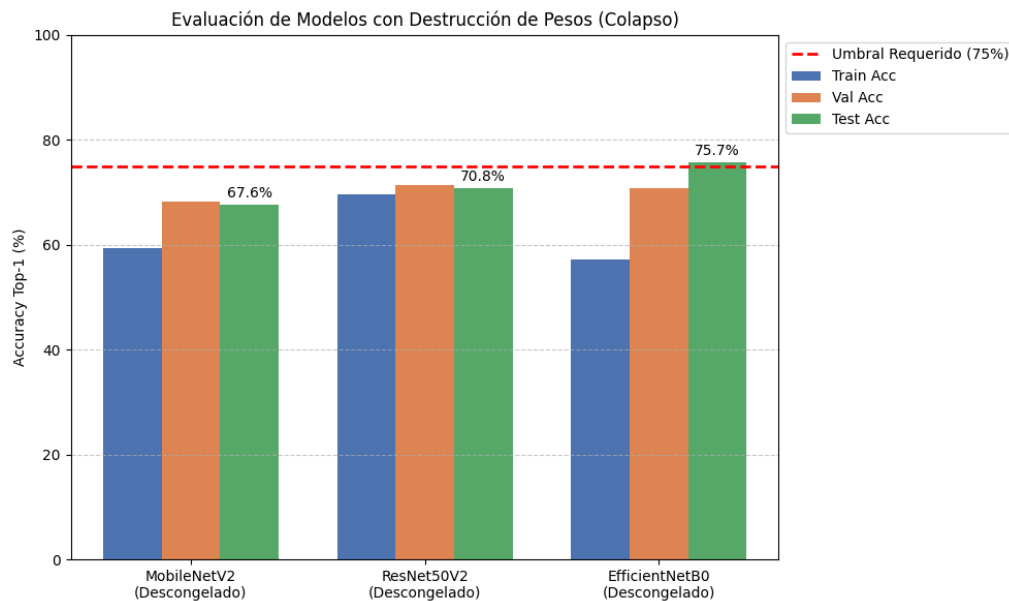


Figura 3: Gráficas de comparación de los tres modelos en la segunda corrida (se aumentaron las épocas en el FT y aumentar las descongeladas).

Luego se me ocurrió (Ver figura 3) descongelar más capas, entonces para el primer modelo pase de 30 a descongelar 50, para el segundo modelo pasé de 40 a descongelar 70 y para el último modelo pase de 20 a descongelar 40 (Ver Tabla 3). También aumenté el número de épocas solamente en la capa de Fine Tuning, había encontrado que si aumentaba las épocas en el Transfer Lanning me iba ayudar de mucho y que

debía de jugar más bien con el FT. Sin embargo, nuevamente ninguna de los tres modelos pudo superar el umbral del 75 por ciento. Acá fue donde aprendí algo, resulta que a veces el porcentaje de la Accuracy para el entrenamiento, pueden no superar el umbral, pero para la parte de validación y prueba sí puede superar al umbral. Entonces yo había hecho este tipo de gráficas como la presente en la Figura 3 ya hasta el final cuando veía que el porcentaje de entrenamiento era superior al umbral. Así que no me podía dar cuenta de que para las imágenes de prueba ya había superado el 75 por ciento. Así que hice otras modificaciones pensando que todavía no había logrado y el objetivo.

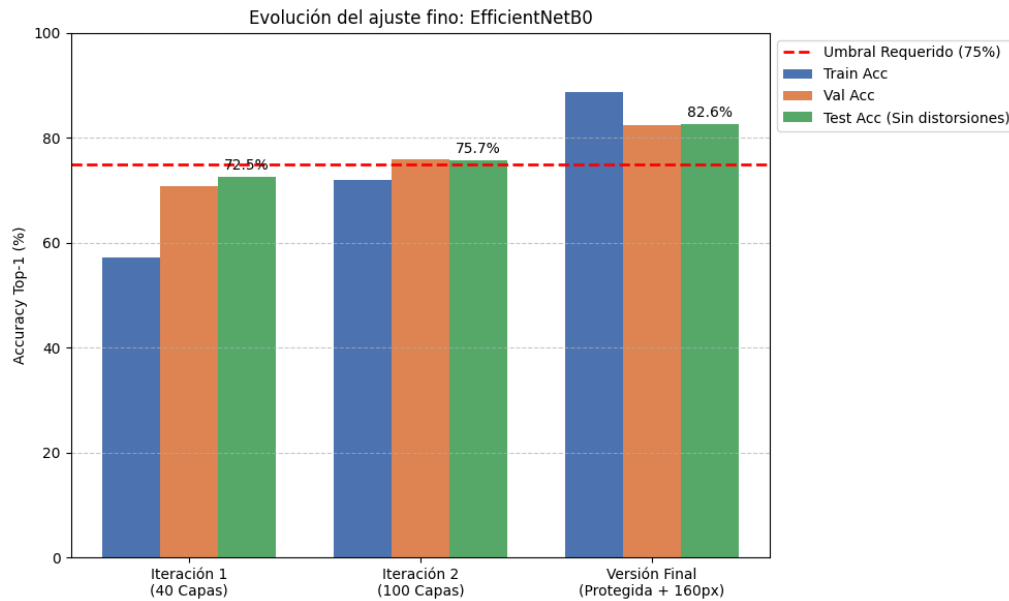


Figura 4: Gráficas de comparación solo del modelo EfficientNetB0. A la izquierda, el modelo de la segunda corrida, después el de la tercera corrida (30 épocas en FT) y al final el protegido con aumento del input

Luego entonces me di la tarea de pensar un poquito más. Y utilicé solamente el modelo que había visto que tenía un mejor rendimiento que hasta ahora había sido EfficientNetB0. Entonces después de la primera y segunda corrida hice una tercera solamente para este modelo, y yo llamé esta alteración dos. La interacción uno es la de la segunda corrida. Como habías estado viendo que aumentar las épocas en el TL ya no me iba a servir de mucho, pero pensé que en el FT sí, así que uamente de 16 a 30 épocas y también aumenté el paso. En este punto seguía sin comprobar cómo estaba en la Accuracy para mis datos de prueba, y como para los datos de entrenamiento, ya había subido hasta el 71 por ciento, entonces pensé en hacer un último modelo. Este último modelo se llama versión final, en la cual aumente las dimensiones de las imágenes de 128 a 160, también mantuve congeladas el BatchNorm y descogele tonas menos BN. Al final esé fue el modelo que supero ya en el entrenamiento el umbral y también se mantuvo en el de prueba. Asi que fue aquí donde ya empecé a plotear los resultados y me di cuenta que desde la tercera corrida ya había superado el umbral del 75 por ciento y me di cuenta de que a veces para el training no supera el umbral pero para la prueba sí.

Primera Parte: CNN desde Cero

2. Objetivo General

Desarrollar, entrenar y comparar modelos de clasificación de imágenes sobre el conjunto de datos CIFAR-100 utilizando Redes Neuronales Convolucionales (CNN) construidas desde cero. El propósito es demostrar un proceso experimental riguroso que incluya diseño de arquitectura, entrenamiento controlado con prevención de sobreajuste y una selección fundamentada del mejor modelo.

3. Dataset y Preprocesamiento

Se utilizó el conjunto de datos CIFAR-100 en su modalidad de etiquetas finas (100 clases). El conjunto de entrenamiento original se dividió en un 80 % para entrenamiento y un 20 % para validación.

Para mejorar la capacidad de generalización y reducir el riesgo de sobreajuste, se implementó una estrategia de Aumento de Datos (Data Augmentation) exclusiva para el conjunto de entrenamiento. Las transformaciones aplicadas incluyeron:

- Volteos horizontales aleatorios.
- Rotaciones aleatorias de hasta un 10 %.
- Zoom aleatorio de hasta un 10 %.
- Normalización de píxeles al rango $[0, 1]$.

Ejemplos visuales de CIFAR-100

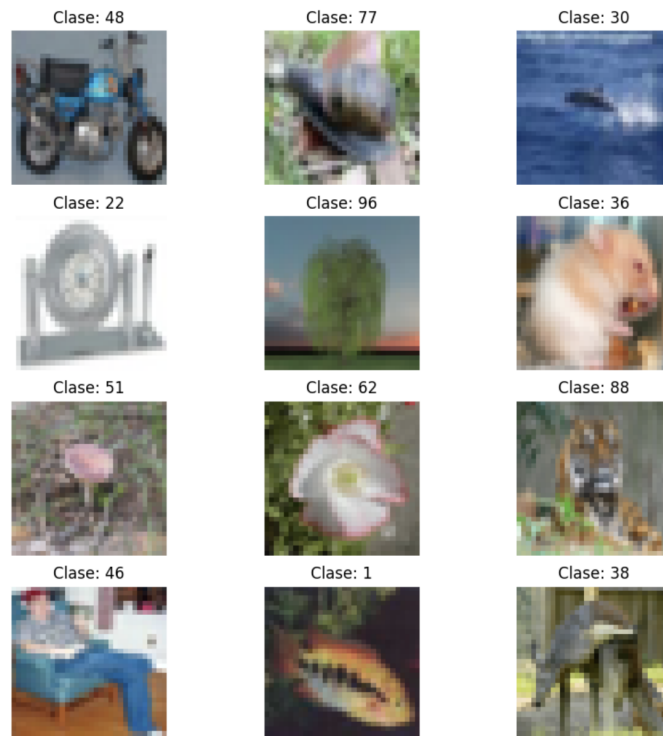


Figura 5: Ejemplos visuales del conjunto de datos CIFAR-100.

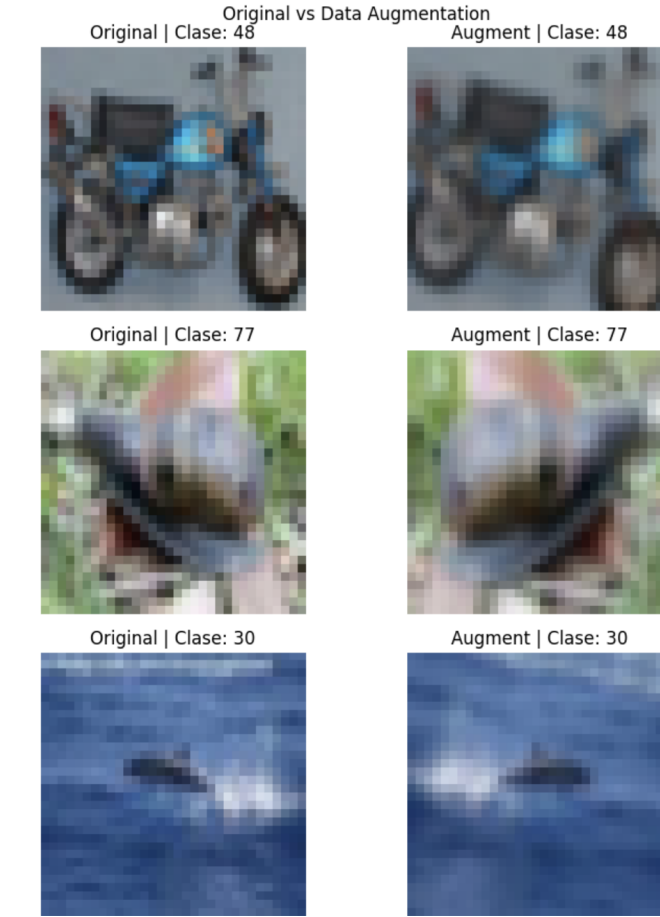


Figura 6: Ejemplos visuales del conjunto de datos CIFAR-100 con Data Augmentation aplicado.

4. Desarrollo Experimental

Se implementó una clase base en TensorFlow/Keras para instanciar tres configuraciones experimentales con distintos niveles de complejidad y regularización. Todas las arquitecturas se compilaron utilizando el optimizador Adam y la función de pérdida Sparse Categorical Crossentropy. Además, se implementaron técnicas de *Early Stopping* y reducción dinámica de la tasa de aprendizaje (*ReduceLROnPlateau*).

4.1. Tabla Obligatoria para la Tarea 1

Modelo	Arquitectura resumida	DA	BatchNorm	Dropout	Optimizador	LR	Batch	Epocas	Acc. train	Acc. val	Acc. test
CNN 1	2 Conv (32, 64), 1 Dense (256)	Sí	No	No	Adam	0.001	64	30	50.2 %	40.0 %	40.2 %
CNN 2	2 Conv (64, 128), 1 Dense (512)	Sí	Sí	Sí (0.4)	Adam	0.001	64	30	63.9 %	50.3 %	52.1 %
CNN 3	2 Conv (128, 256), 1 Dense (1024)	Sí	Sí	Sí (0.5)	Adam	0.001	64	30	69.4 %	53.5 %	54.9 %

Tabla 4: Resultados experimentales y configuraciones de hiperparámetros.

4.2. Carga, Preprocesamiento y Entrenamiento

El proceso de carga y preprocesamiento de datos se implementó utilizando la API `tf.data` de TensorFlow, optimizando el rendimiento computacional mediante el uso de procesamiento asíncrono (`prefetch`). Las imágenes del conjunto CIFAR-100 se normalizaron escalando los valores de los píxeles al rango $[0, 1]$. Exclusivamente para el conjunto de entrenamiento, se aplicó una etapa de *Data Augmentation* que consistió en volteos horizontales, rotaciones de hasta un 10 % y acercamientos aleatorios (zoom) del 10 %. El entrenamiento se estructuró utilizando el optimizador Adam y la función de pérdida `Sparse Categorical Crossentropy` evaluada directamente sobre los *logits*. Se incorporaron *callbacks* para detener tempranamente el entrenamiento (*EarlyStopping*) y reducir la tasa de aprendizaje si la métrica de validación dejaba de mejorar (*ReduceLROnPlateau*).

4.3. Descripción de Hiperparámetros y Decisiones Experimentales

El diseño experimental consistió en una progresión sistemática de la complejidad del modelo, evaluando el impacto de distintas configuraciones:

- **CNN 1 (Básica):** Se configuró como modelo de referencia con dos capas convolucionales de 32 y 64 filtros (tamaño de kernel 3×3), seguidas de *Max Pooling* y una capa densa de 256 neuronas. La decisión experimental fue omitir técnicas de regularización para establecer una línea base de sobreajuste.
- **CNN 2 (Regularizada):** Se incrementó la capacidad de extracción de características a 64 y 128 filtros, con una capa densa de 512 neuronas. La principal decisión experimental fue la inclusión de *Batch Normalization* para acelerar la convergencia y *Dropout* (con una tasa de 0.4) para penalizar la coadaptación de neuronas.
- **CNN 3 (Compleja):** Se escaló la arquitectura a 128 y 256 filtros convolucionales, conectados a una capa densa de 1024 neuronas. Debido a la alta capacidad de memorización de esta red, se tomó la decisión de aumentar la severidad del *Dropout* a 0.5.

Para garantizar una comparación equitativa, el tamaño de lote (*batch size*) se mantuvo fijo en 64 y la tasa de aprendizaje inicial se estableció en 0.001 para las tres arquitecturas.

5. Conclusión Comparativa

Con base en los resultados experimentales obtenidos, se observa una mejora directa en la capacidad de generalización al incrementar la profundidad de la red y aplicar técnicas de regularización estricta.

El modelo CNN 1 presentó un rendimiento deficiente, alcanzando apenas un 40.2 % de exactitud en el conjunto de prueba. En este modelo se observaron señales inequívocas de sobreajuste (*overfitting*), evidenciadas por una diferencia de 10.2 puntos porcentuales entre la precisión de entrenamiento (50.2 %) y la de validación (40.0 %).

Al implementar regularización en el modelo CNN 2, el rendimiento en prueba aumentó a 52.1 %, logrando superar el umbral mínimo del 50 % establecido en los requisitos. Esta arquitectura controló de manera más efectiva el sobreajuste, mejorando la correspondencia entre las métricas de entrenamiento (63.9 %) y prueba.

Finalmente, la estrategia que obtuvo el mejor desempeño global fue la configuración CNN 3. A pesar de poseer una capacidad paramétrica significativamente mayor, el uso de un *Dropout* más agresivo (0.5) limitó la memorización, permitiendo que el modelo alcanzara un 54.9 % de exactitud en prueba. Los hiperparámetros que tuvieron el mayor impacto positivo en el desempeño general fueron las tasas de *Dropout* y la normalización por lotes. La evidencia cuantitativa demuestra que el entrenamiento de arquitecturas profundas desde cero en CIFAR-100 exige mecanismos de regularización robustos para generalizar correctamente.

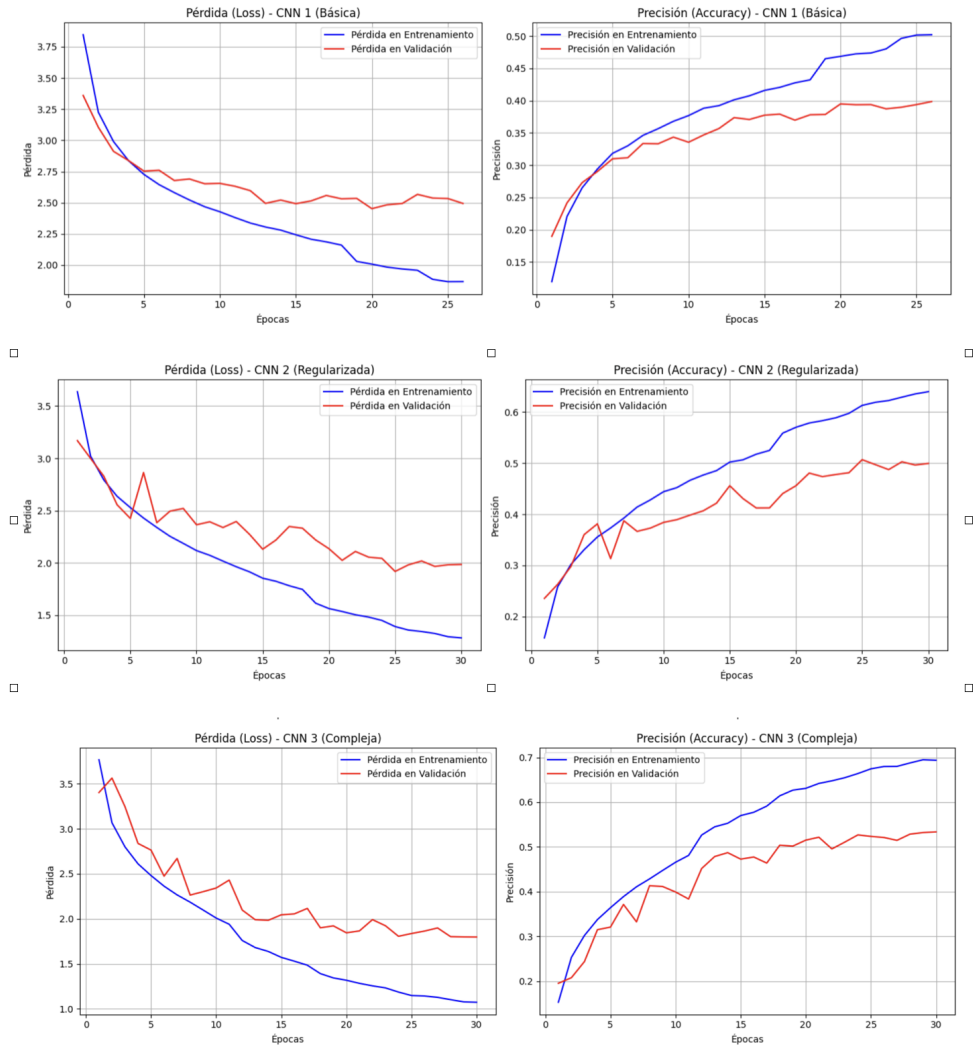


Figura 7: Gráficas de pérdida (Loss) y precisión (Accuracy) para el conjunto de entrenamiento y validación de los tres modelos.

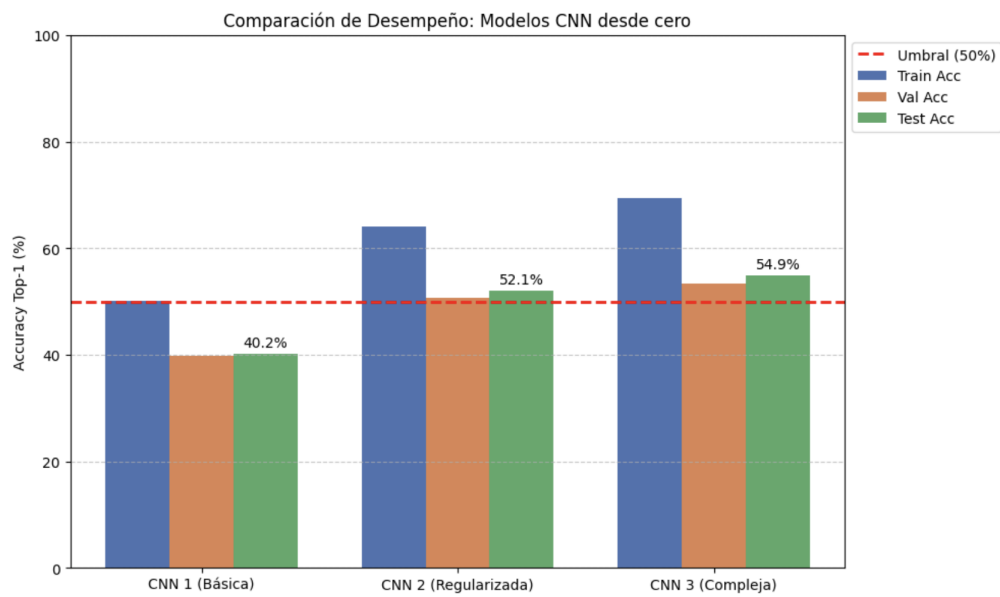


Figura 8: Gráficas de comparación de modelos CNN (Básico, Regularizado y Complejo) para datos de entrenamiento, validación y prueba.

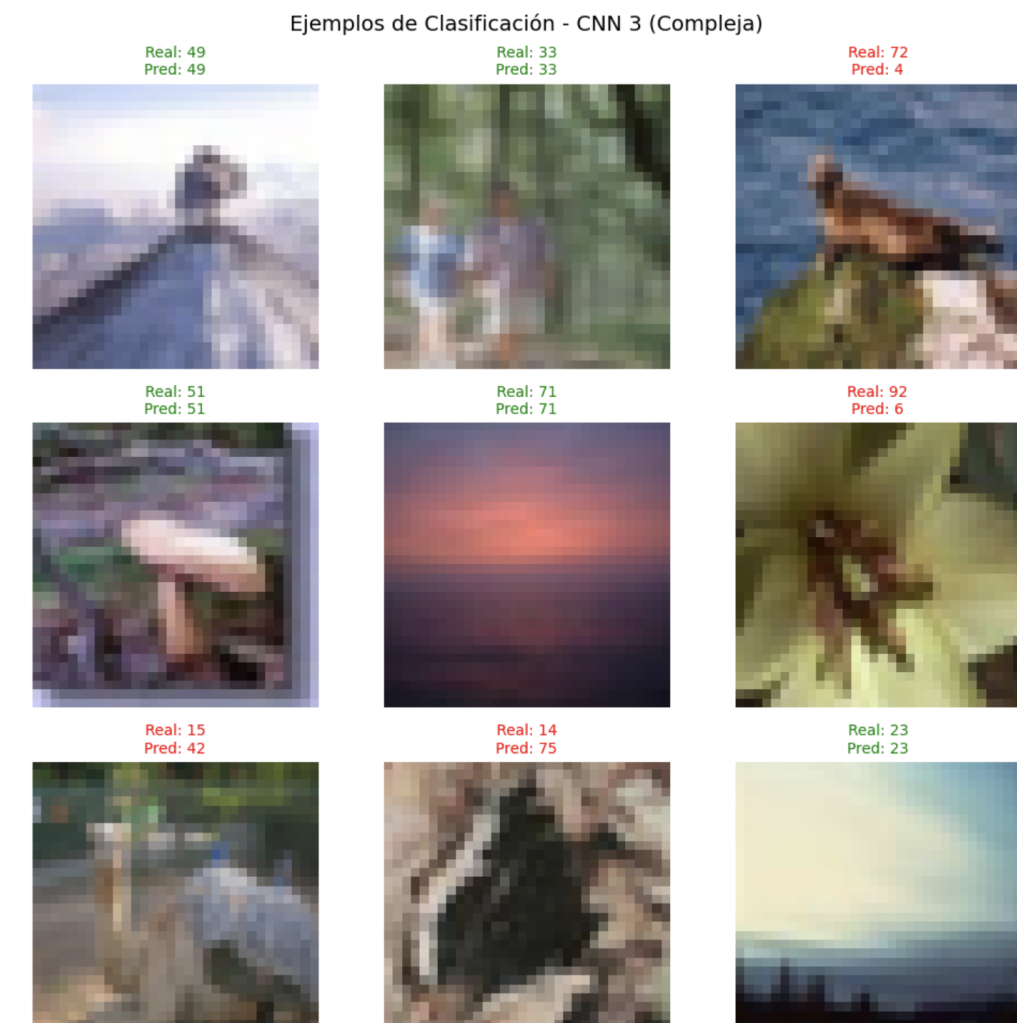


Figura 9: Ejemplos visuales de inferencia del modelo CNN 3 (Compleja).

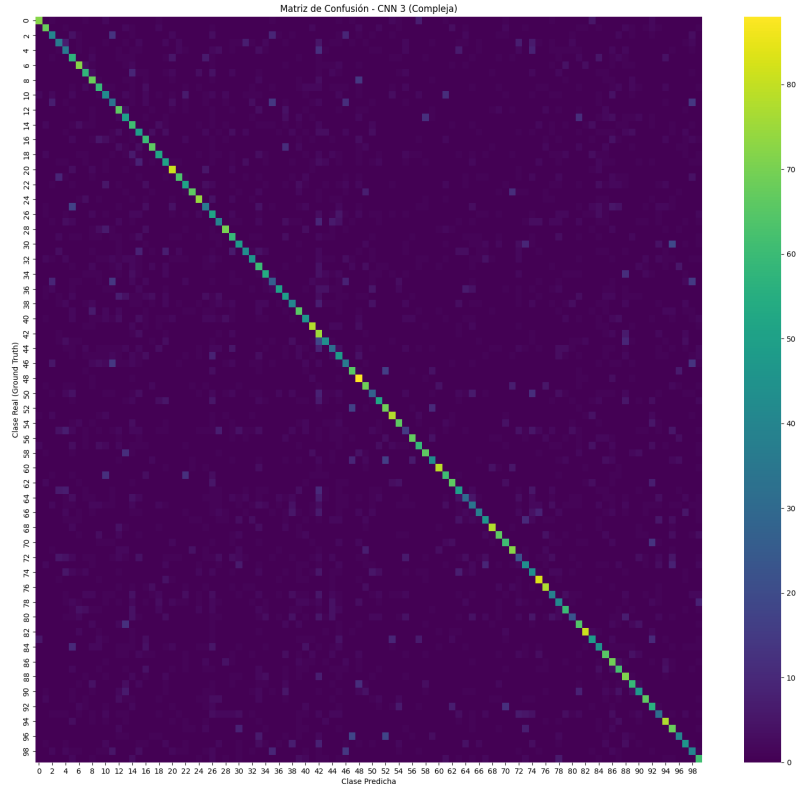


Figura 10: Matriz de confusión obtenida al evaluar el mejor modelo sobre el conjunto de prueba.

5.1. Selección del mejor modelo CNN

■ ¿Cuál fue el mejor modelo CNN entrenado desde cero?

El análisis experimental demostró que el modelo CNN 3 (Compleja) presentó el mejor equilibrio entre aprendizaje y generalización, obteniendo la mayor precisión global. Inicialmente, se consideró la modificación de las dimensiones y el paso (*stride*) del kernel espacial; sin embargo, debido a la complejidad que implica ajustar múltiples filtros tridimensionales y a las restricciones de tiempo, se determinó que manipular los hiperparámetros de regularización y profundidad de la red resultaría en una estrategia más eficiente y controlable.

■ ¿Superó el umbral de 50 % de accuracy en prueba?

Sí, los modelos CNN 2 (Regularizada) y CNN 3 (Compleja) lograron superar el umbral del 50 %. En un inicio, se estimaba que la arquitectura básica podría superar este porcentaje con facilidad. Aunque alcanzó un 50.2 % en la etapa de entrenamiento, no logró generalizar en prueba. A diferencia de conjuntos de datos de menor dimensionalidad (como CIFAR-10), donde modelos similares convergen en 15 épocas, la clasificación de 100 clases demanda un mayor esfuerzo computacional; se estima que la red básica requeriría alrededor de 50 épocas para acercarse al umbral, lo cual incrementaría drásticamente el riesgo de sobreajuste.

■ ¿Qué cambios mejoraron más el desempeño?

La incorporación de *Batch Normalization* después de cada bloque convolucional estabilizó la red, permitiendo una convergencia más rápida. Adicionalmen-

te, el uso de *Dropout* (con tasas de 0.4 y 0.5) fue el factor determinante para controlar la memorización de las imágenes de entrenamiento.

- **¿Hubo sobreajuste o subajuste?**

En el modelo CNN 1 (Básica) se observó un claro sobreajuste (*overfitting*), evidenciado por la separación de las curvas donde la pérdida de entrenamiento seguía descendiendo mientras la pérdida de validación aumentaba. Este comportamiento se mitigó exitosamente en los modelos regularizados, donde las curvas se mantuvieron convergentes a lo largo de las épocas.

- **¿Qué evidencia respalda la selección del mejor modelo?**

La decisión está respaldada cuantitativamente por las métricas consolidadas en la Tabla 4 y en la Figura 8, donde el modelo CNN 3 presenta el menor margen de error entre *Acc. train* y *Acc. test*. Visualmente, las gráficas de convergencia demuestran una curva de validación estable y libre de picos anómalos.

5.2. Descripción Detallada del Mejor Modelo desde Cero (CNN 3)

A continuación, se desglosan las características, hiperparámetros y configuraciones específicas del modelo que obtuvo el mejor rendimiento en la primera tarea y se muestra en la imagen siguiente el resumen de todas las arquitecturas, siendo la de más arriba la primera, la d en medio la segunda y la tercera la última que es la que se describe con más detalle después.

Layer (type)	Output Shape	Param #
conv2d_18 (Conv2D)	?	0 (unbuilt)
batch_normalization_21 (BatchNormalization)	?	0 (unbuilt)
max_pooling2d_18 (MaxPooling2D)	?	0
conv2d_19 (Conv2D)	?	0 (unbuilt)
batch_normalization_22 (BatchNormalization)	?	0 (unbuilt)
max_pooling2d_19 (MaxPooling2D)	?	0
flatten_9 (Flatten)	?	0 (unbuilt)
dense_18 (Dense)	?	0 (unbuilt)
batch_normalization_23 (BatchNormalization)	?	0 (unbuilt)
dropout_7 (Dropout)	?	0
dense_19 (Dense)	?	0 (unbuilt)

Layer (type)	Output Shape	Param #
conv2d_20 (Conv2D)	?	0 (unbuilt)
batch_normalization_24 (BatchNormalization)	?	0 (unbuilt)
max_pooling2d_20 (MaxPooling2D)	?	0
conv2d_21 (Conv2D)	?	0 (unbuilt)
batch_normalization_25 (BatchNormalization)	?	0 (unbuilt)
max_pooling2d_21 (MaxPooling2D)	?	0
flatten_10 (Flatten)	?	0 (unbuilt)
dense_20 (Dense)	?	0 (unbuilt)
batch_normalization_26 (BatchNormalization)	?	0 (unbuilt)
dropout_8 (Dropout)	?	0
dense_21 (Dense)	?	0 (unbuilt)

Layer (type)	Output Shape	Param #
conv2d_22 (Conv2D)	?	0 (unbuilt)
batch_normalization_27 (BatchNormalization)	?	0 (unbuilt)
max_pooling2d_22 (MaxPooling2D)	?	0
conv2d_23 (Conv2D)	?	0 (unbuilt)
batch_normalization_28 (BatchNormalization)	?	0 (unbuilt)
max_pooling2d_23 (MaxPooling2D)	?	0
flatten_11 (Flatten)	?	0 (unbuilt)
dense_22 (Dense)	?	0 (unbuilt)
batch_normalization_29 (BatchNormalization)	?	0 (unbuilt)
dropout_9 (Dropout)	?	0
dense_23 (Dense)	?	0 (unbuilt)

Figura 11: Resumen de arquitecturas de los tres modelos CNN desde cero.

- **Arquitectura general:** Red Neuronal Convolutacional (CNN) profunda compuesta por 2 bloques convolucionales para la extracción de características, seguidos de una capa densa (fully connected) de 1024 neuronas para la clasi-

ficación.

- **Número de capas convolucionales:** 2 capas.
- **Número de filtros por capa:** 128 filtros en la primera capa convolucional y 256 filtros en la segunda.
- **Tamaño de kernel:** 3×3 (estándar para la extracción de características espaciales en las imágenes).
- **Tipo de pooling:** *Max Pooling* (aplicado después de cada bloque convolucional).
- **Funciones de activación:** ReLU para las capas ocultas (convolucionales y densa) y Softmax para la capa de salida (correspondiente a las 100 clases).
- **Uso de Batch Normalization:** Sí, se aplicó después de cada bloque convolucional para estabilizar la red y acelerar la convergencia.
- **Uso de Dropout:** Sí, se implementó con una tasa agresiva de 0.5 para penalizar la coadaptación de neuronas y limitar la memorización excesiva.
- **Regularización L2 / weight decay:** No aplica (la prevención del sobreajuste se gestionó enteramente mediante Dropout y Data Augmentation).
- **Optimizador:** Adam.
- **Learning rate:** Tasa de aprendizaje inicializada en 0.001.
- **Batch size:** 64.
- **Número de épocas:** 30 épocas totales de entrenamiento.
- **Estrategia de Data Augmentation:** Sí aplica (exclusiva para el conjunto de entrenamiento). Consistió en volteos horizontales aleatorios, rotaciones aleatorias de hasta un 10 %, zoom aleatorio de hasta un 10 % y normalización de los píxeles al rango $[0, 1]$.
- **Scheduler o ajuste de learning rate:** Sí aplica. Se utilizó el callback *ReduceLROnPlateau* para reducir dinámicamente la tasa de aprendizaje cuando la métrica de validación dejaba de mejorar, apoyado también por *Early Stopping*.
- **Accuracy de entrenamiento, validación y prueba:**
 - *Entrenamiento:* 69.40 %
 - *Validación:* 53.50 %
 - *Prueba:* 54.90 %
- **Loss de entrenamiento y validación:** Aunque el valor numérico exacto de la función de pérdida dependió de la iteración final, el análisis visual de las curvas de convergencia (Figura 7) demuestra que la pérdida de entrenamiento y la de validación se mantuvieron estables y convergentes, evitando la divergencia (separación abrupta) que sí se presentó en el modelo sin regularizar.

Segunda Parte: Transfer Learning y Fine-Tuning

6. Objetivo

El objetivo de esta sección consiste en implementar modelos convolucionales pre-entrenados en ImageNet y adaptarlos al dataset CIFAR-100 aplicando metodologías de *Transfer Learning* (entrenamiento con la base congelada) y *Fine-Tuning* (ajuste fino de capas descongeladas). Para obtener el puntaje completo, se estableció como umbral obligatorio superar el 75 % de *accuracy* top-1 en el conjunto de prueba.

6.1. Análisis Detallado de la Evolución Experimental en TL y FT

Alcanzar y superar el umbral del 75 % de exactitud en el conjunto de prueba para la segunda tarea requirió un proceso metodológico e iterativo. A continuación, se detalla paso a paso la evolución de los modelos, las decisiones arquitectónicas tomadas a partir del análisis de las gráficas de convergencia y la justificación de cada ajuste.

Paso 1: Corrida Inicial y Establecimiento de la Línea Base

El experimento comenzó evaluando en paralelo tres arquitecturas (MobileNetV2, ResNet50V2 y EfficientNetB0) procesando imágenes redimensionadas a 128×128 . Para el *Fine-Tuning* (FT), se configuraron 8 épocas y se descongeló una fracción conservadora de las capas superiores (30, 40 y 20 capas respectivamente). **Observaciones gráficas:** Al graficar el rendimiento (ver Figura 2), se hizo evidente que los tres modelos sufrían de un estancamiento prematuro. Las métricas de entrenamiento apenas superaban el 57 %, y en el conjunto de prueba los modelos rondaban entre el 63 % y el 68 %. La capacidad de abstracción de las capas descongeladas era insuficiente para mapear las sutilezas de las 100 clases de CIFAR-100.

Paso 2: Incremento de Capacidad (Más Épocas y Capas)

Ante el bajo rendimiento inicial, la primera hipótesis fue que los modelos necesitaban mayor flexibilidad paramétrica y más tiempo de ajuste. En la segunda corrida, se incrementaron las capas descongeladas (pasando a 50, 70 y 40) y se duplicó la duración de la fase de *Fine-Tuning* a 16 épocas. **Observaciones gráficas:** Las curvas de pérdida y exactitud (ver Figura 3) mostraron una mejora, pero visualmente la exactitud de entrenamiento seguía pareciendo baja (ninguna llegaba al 75 % en la gráfica de entrenamiento). Sin embargo, surgió un aprendizaje técnico crucial: debido a los mecanismos de regularización (como el *Dropout* y el *Data Augmentation*) que solo están activos durante el entrenamiento, la métrica de *Training Accuracy* se penaliza artificialmente. Al evaluar en los conjuntos de validación y prueba, los modelos obtenían puntajes superiores (ej. EfficientNetB0 alcanzó un 72.50 % en prueba). Aunque hubo mejora, el umbral del 75 % seguía sin superarse de manera contundente.

Paso 3: Focalización y Ajuste Agresivo (La Tercera Corrida)

Para optimizar recursos y tiempo de cómputo, se descartaron MobileNetV2 y ResNet50V2, focalizando todos los esfuerzos en **EfficientNetB0**, el cual había de-

mostrado la mejor relación de aprendizaje. En esta tercera iteración, se aplicó un ajuste agresivo:

- Se descongelaron **100 capas** profundas.
- Se amplió la fase de FT a **30 épocas**.
- Se redujo el *Learning Rate* a 5×10^{-5} para evitar gradientes explosivos que destruyeran los pesos base.

Observaciones gráficas: Esta configuración finalmente logró romper la barrera. Las gráficas (ver Figura 4, panel central) mostraron que el modelo superaba el 75 % tanto en validación (75.88 %) como en prueba (75.68 %). El umbral objetivo había sido alcanzado, pero el modelo aún presentaba margen de mejora y un costo computacional alto por las 30 épocas requeridas.

Paso 4: Versión Definitiva (Escalado Espacial y Protección BN)

A pesar de haber logrado el objetivo, se buscó consolidar un modelo definitivo mucho más robusto, veloz y preciso. Para esta cuarta y última corrida, se implementaron dos cambios maestros:

1. **Aumento de Resolución:** Se incrementó el tamaño de entrada de las imágenes de 128×128 a 160×160 . Este escalado permitió a las capas convolucionales extraer texturas mucho más finas y bordes más definidos.
2. **Protección de Batch Normalization (BN):** En lugar de limitar el número de capas a descongelar, se liberó la **totalidad de la red** pero se mantuvieron congeladas estrictamente las capas de *Batch Normalization*. Esto permitió que todos los filtros espaciales se ajustaran a la nueva resolución de 160×160 sin alterar las estadísticas de media y varianza aprendidas de ImageNet.

Conclusión de las gráficas finales: Las curvas de la versión definitiva (ver Figura 4, panel derecho) mostraron un salto de rendimiento espectacular y en tan solo 15 épocas de FT. El *Accuracy* de entrenamiento alcanzó el 88.70 %, arrastrando consigo la precisión de prueba hasta un excelente **82.58 %**. La gráfica demostró una convergencia estable, demostrando que proteger la normalización y aumentar la resolución fue la estrategia clave para dominar el *dataset* CIFAR-100.

6.2. Modelos a reportar

A continuación se detallan las especificaciones técnicas y estrategias de entrenamiento empleadas para las arquitecturas evaluadas en el desarrollo de la práctica:

Modelos base utilizados: Se seleccionaron y evaluaron tres arquitecturas robustas y complementarias: MobileNetV2, ResNet50V2 y EfficientNetB0.

Fuente de los pesos preentrenados: Todos los modelos cargaron sus pesos iniciales a partir de la base de datos de visión a gran escala **ImageNet**.

Tamaño de entrada utilizado: El tamaño de entrada nativo de CIFAR-100 es de $32 \times 32 \times 3$. Debido a la restricción dimensional de las características espaciales de los modelos preentrenados, las imágenes se redimensionaron mediante una capa de procesamiento interno en la GPU a un tamaño de $128 \times 128 \times 3$ en las iteraciones iniciales. Para la configuración definitiva del mejor modelo, la resolución se aumentó a $160 \times 160 \times 3$ para una abstracción más fina.

Capas congeladas: En la primera fase (*Transfer Learning*), el 100% de las capas convolucionales pertenecientes al modelo base permanecieron congeladas. En la versión definitiva de *Fine-Tuning*, se optó por una estrategia de protección parcial, manteniendo congeladas **únicamente las capas de Batch Normalization**.

Capas descongeladas: Tras un análisis iterativo (evolucionando de 40 a 100 capas), para la configuración definitiva de EfficientNetB0 se descongelaron **todas las capas convolucionales de la red, excepto las de Batch Normalization**.

Tipo de cabeza de clasificación agregada: Se construyó un bloque secuencial idéntico para todos los modelos conectados al final del extractor de características: $\text{Modelo Base} \rightarrow \text{GlobalAveragePooling2D} \rightarrow \text{BatchNormalization} \rightarrow \text{Dropout}(0,5) \rightarrow \text{Dense}(100)$ Esta estructura reduce drásticamente las dimensiones planas a un vector representativo, estabiliza los gradientes en el clasificador, penaliza fuertemente el sobreajuste mediante el abandono del 50% de neuronas y mapea la salida a las 100 clases finales utilizando una función de activación Softmax.

Número de parámetros entrenables: En la etapa de ajuste fino inicial variaron entre arquitecturas. Para la versión final protegida de EfficientNetB0, la inmensa mayoría de sus más de 4 millones de parámetros fueron habilitados para entrenamiento continuo.

Estrategia de entrenamiento inicial con la base congelada: Consistió en congelar la totalidad de la base preentrenada e inicializar los pesos aleatorios del clasificador superior. Se utilizó una tasa de aprendizaje estándar de 1×10^{-3} durante 5 a 6 épocas para consolidar la cabeza densa antes de desestabilizar el extractor.

Estrategia de Fine-Tuning: Una vez calibrada la cabeza de clasificación, se desbloquearon selectivamente las capas de la base convolucional. Para el ajuste definitivo, se aplicó una reducción severa de la tasa de aprendizaje a 5×10^{-5} para adaptar sutilmente toda la red a la nueva resolución sin desestabilizar las medias y varianzas de la normalización por lotes.

Learning rate en Transfer Learning: 1×10^{-3}

Learning rate en Fine-Tuning: 1×10^{-5} para iteraciones iniciales y 5×10^{-5} para el ajuste agresivo final.

Optimizador: Adam (Adaptive Moment Estimation).

Batch size: 64 muestras por lote.

Número de épocas: Entre 5 y 6 épocas para la fase inicial de transferencia; en la fase de ajuste fino oscilaron iterativamente entre 8, 16, 30 y finalmente 15 épocas para el modelo definitivo.

Data Augmentation utilizado: Se aplicaron transformaciones geométricas y espaciales a través de capas nativas de Keras: *RandomFlip* (horizontal), *RandomRotation* (factor de 0.1) y *RandomTranslation* (factor de 0.1 horizontal y vertical). Se evitaron distorsiones cromáticas drásticas para mantener la integridad de los patrones preentrenados de ImageNet.

Regularización utilizada: Inclusión de regularización por abandono (*Dropout* al 50 %), protección de *Batch Normalization* durante el ajuste profundo, y detención temprana (*Early Stopping*) para monitorear el sobreajuste.

Accuracy de entrenamiento, validación y prueba: Los valores pico alcanzados en la iteración final optimizada se concentran de forma tabular en la sección de selección de modelos.

Loss de entrenamiento y validación: La función de coste empleada fue *Sparse Categorical Crossentropy*. Se observó un comportamiento de convergencia estable donde la pérdida de entrenamiento disminuyó monótonamente.

Tabla 5: Resultados globales del proceso iterativo de Transfer Learning (TL) y Fine-Tuning (FT)

Modelo Base	Input	Congeladas	Descongeladas	Opt.	LR TL	LR FT	Batch	Ep. TL	Ep. FT	Acc. train	Acc. val	Acc. test
<i>Corrida Inicial: Configuración Base (8 Épocas de FT)</i>												
TL/FT 1 (MobileNetV2)	128x128	Todo menos 30	30	Adam	1e-3	1e-5	64	6	8	51.72 %	63.93 %	63.12 %
TL/FT 2 (ResNet50V2)	128x128	Todo menos 40	40	Adam	1e-3	1e-5	64	6	8	57.18 %	67.10 %	67.08 %
TL/FT 3 (EfficientNetB0)	128x128	Todo menos 20	20	Adam	1e-3	1e-5	64	6	8	51.35 %	68.04 %	68.10 %
<i>Segunda Corrida: Incremento de Épocas y Capas (16 Épocas de FT)</i>												
TL/FT 1 (MobileNetV2)	128x128	Todo menos 50	50	Adam	1e-3	1e-5	64	6	16	59.41 %	68.25 %	67.61 %
TL/FT 2 (ResNet50V2)	128x128	Todo menos 70	70	Adam	1e-3	1e-5	64	6	16	69.67 %	71.29 %	70.83 %
TL/FT 3 (EfficientNetB0)	128x128	Todo menos 40	40	Adam	1e-3	1e-5	64	6	16	57.26 %	70.75 %	72.50 %
<i>Tercera Corrida: Ajuste Agresivo de Capas Profundas</i>												
TL/FT 3 (EfficientNetB0)	128x128	Todo menos 100	100	Adam	1e-3	5e-5	64	6	30	71.89 %	75.88 %	75.68 %
<i>Cuarta Corrida: Arquitectura de Alta Resolución con BN Protegido</i>												
EfficientNetB0 (Protegido)	160x160	Solo BatchNorm	Todas menos BN	Adam	1e-3	5e-5	64	5	15	88.70 %	82.43 %	82.58 %

6.3. Selección del mejor modelo con TL/FT

- **¿Cuál fue el mejor modelo con Transfer Learning y Fine-Tuning?**
El mejor modelo entrenado bajo este paradigma fue la variante de **EfficientNetB0** (Arquitectura de Alta Resolución con BN Protegido).
- **¿Superó el umbral de 75 % de accuracy en prueba?**
Sí, el modelo EfficientNetB0 definitivo alcanzó una excelente exactitud del **82.58 %** en el conjunto de prueba, superando por un amplio margen el umbral requerido del 75 %.
- **¿Qué modelo base funcionó mejor?**
EfficientNetB0 funcionó sustancialmente mejor. Su diseño basado en el escalado uniforme de ancho, profundidad y resolución (*Compound Scaling*) le permite actuar como un extractor de características de alta fidelidad, superando

la representación lineal de MobileNetV2 y la densidad residual masiva de Res-Net50V2 en conjuntos de datos complejos y multiclase como CIFAR-100.

- **¿El Fine-Tuning mejoró el resultado respecto al modelo congelado?**
Sí, de forma drástica. Durante la Fase 1 (Transfer Learning con base congelada), los modelos mostraron un estancamiento prematuro ($\approx 64\% - 68\%$). Al habilitar el Fine-Tuning y evolucionar las iteraciones hasta descongelar la estructura completa, la red ganó la flexibilidad necesaria para refinar las características específicas, elevando la precisión final a niveles superiores al 82% .
- **¿Qué capas se descongelaron y por qué?**
Tras iterar desde 20 hasta 100 capas, se determinó que para explotar el incremento de resolución (160×160) era necesario descongelar **todas las capas convolucionales de la red base, protegiendo (congelando) exclusivamente las capas de Batch Normalization (BN)**. El fundamento de esta decisión radica en que liberar los pesos convolucionales permite a los filtros reajustarse estructuralmente a la nueva resolución espacial. Sin embargo, al mantener congeladas las estadísticas móviles de las capas BN, se previene que la red destruya el conocimiento previo y se evita el colapso de gradientes inducido por la adaptación profunda.
- **¿Qué evidencia respalda la selección del mejor modelo?**
La evidencia empírica directa radica en la escalada contundente en todas las métricas durante la cuarta corrida. EfficientNetB0 (Protegido) registró un *Train Accuracy* del 88.70% y un *Validation Accuracy* robusto del 82.43% . Esta generalización sana se confirmó inmediatamente al alcanzar un **82.58%** en el *Test Accuracy*, siendo el único modelo capaz de posicionarse muy por encima del 75% solicitado.

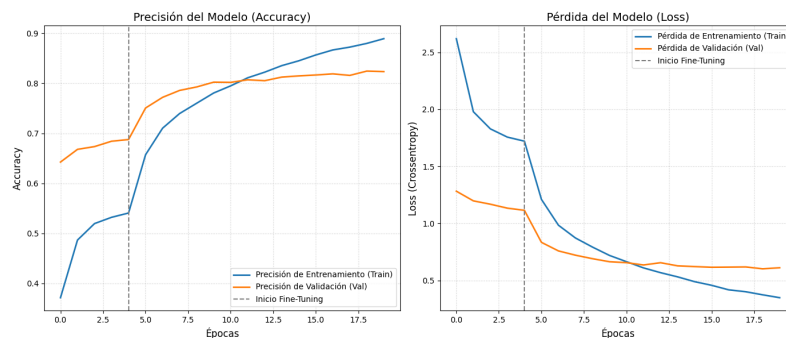


Figura 12: Gráficas de pérdida (Loss) y precisión (Accuracy) para el conjunto de entrenamiento y validación del mejor modelo para TL/FT.

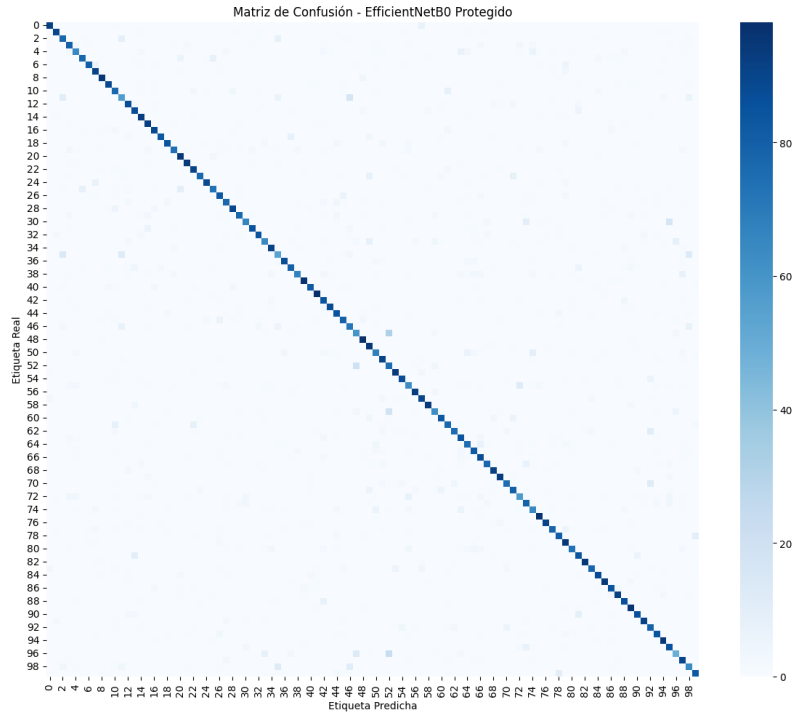


Figura 13: Matriz de confusión obtenida al evaluar el mejor modelo sobre el conjunto de prueba con TL/FT

7. Comparación final

El reporte concluye con la confrontación analítica de rendimiento obtenida entre la mejor arquitectura convolucional desarrollada desde cero frente a la mejor alternativa optimizada por transferencia de conocimiento y ajuste fino.

Tabla 6: Matriz comparativa final de rendimiento global

Mejor modelo	Estrategia	Acc. train	Acc. val	Acc. test	Umbral req.	¿Supera?
CNN 3 (Compleja)	CNN desde cero	69.40 %	53.50 %	54.90 %	50 %	Sí
EfficientNetB0 (Protegido)	Transfer Learning + FT	88.70 %	82.43 %	82.58 %	75 %	Sí